

OnlineCA.jl: A Julia Package for Out-of-core and Sparse Correspondence Analysis

Koki Tsuyuzaki^{1, 2}

¹ Department of Artificial Intelligence Medicine, Graduate School of Medicine, Chiba University, Japan ² Laboratory for Bioinformatics Research, RIKEN Center for Biosystems Dynamics Research, Japan

DOI: [DOIunavailable](#)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Pending Editor](#) 

Reviewers:

- [@Pending Reviewers](#)

Submitted: N/A

Published: N/A

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Correspondence Analysis (CA) is a multivariate technique for the exploratory analysis of contingency tables (Greenacre, 2017; Hill, 1974), widely applied across ecology (Braak, 1986), linguistics and text analysis (Greenacre, 2017), market research (Greenacre, 2017), and more recently single-cell omics (Hsu & Culhane, 2023).

Despite its broad applicability, CA becomes computationally prohibitive for large data matrices, since it requires the singular value decomposition (SVD) of an $N \times M$ matrix. In particular, recent advances in single-cell omics have led to datasets with millions of cells, for which standard CA implementations often fail to scale. To meet this requirement, I present `OnlineCA.jl`, a Julia package for out-of-core and sparse CA (<https://github.com/chiba-ai-med/OnlineCA.jl>). The same out-of-core machinery is reused to support Multiple Correspondence Analysis (MCA) on large categorical tables, including the standard Benz  cri (Benz  cri, 1979) and Greenacre (Greenacre, 2017) eigenvalue corrections.

Statement of need

As the data matrix grows beyond memory, an out-of-core (OOC) implementation — loading only subsets of the data from disk on demand — is desirable. Sparse storage of non-zero entries is additionally advantageous. A CA implementation that supports both is therefore highly desirable (Figure 1).

`OnlineCA.jl` carries the out-of-core, sparse design used by our earlier `OnlinePCA.jl` (Tsuyuzaki et al., 2020; Tsuyuzaki, 2026) and `OnlineNMF.jl` (Tsuyuzaki, 2025) over to CA.

The standardized residual matrix for CA is

$$S = D_r^{-1/2} (P - r_p c_p^\top) D_c^{-1/2},$$

where $P = X/n$ is the relative-frequency matrix, $r_p = P\mathbf{1}$ and $c_p = P^\top\mathbf{1}$ are the row- and column-mass vectors, and $D_r = \text{diag}(r_p)$, $D_c = \text{diag}(c_p)$ are the corresponding diagonal mass matrices. `OnlineCA.jl` never explicitly forms S . Instead, the matrix–vector products Sv and $S^\top u$ are computed by streaming through chunks of X on disk, and the top singular triplets are extracted with a Halko-style randomized SVD (Halko, Martinsson, & Tropp, 2011; Halko, Martinsson, Shkolnisky, et al., 2011). The same implicit operator works uniformly for dense (CSV), sparse (Matrix Market), and binary (Binary COO) inputs.

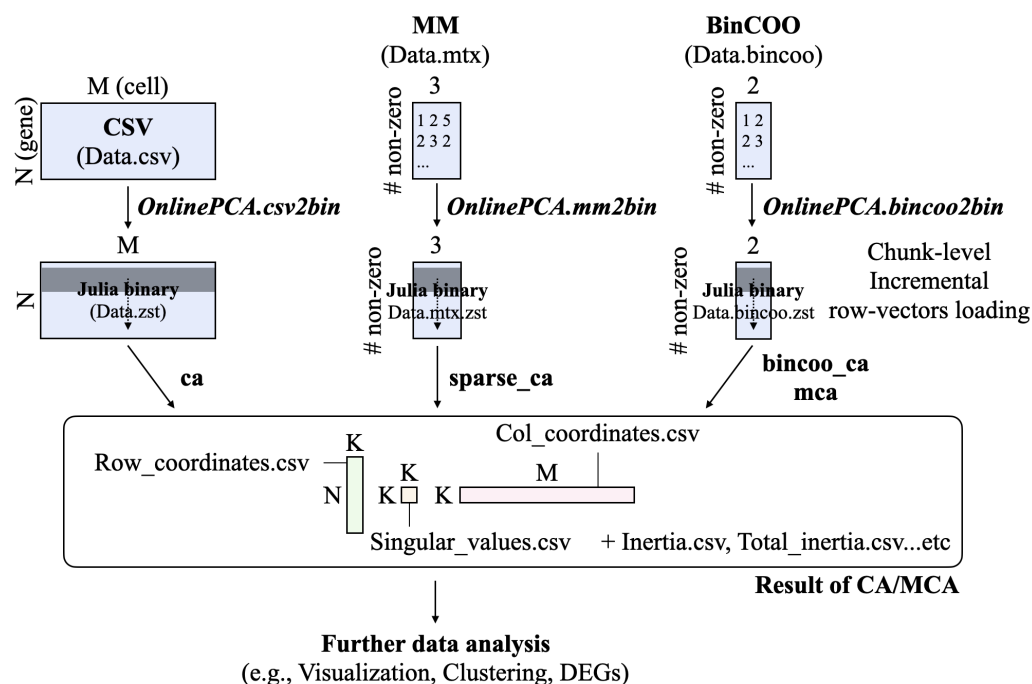


Figure 1: Overview of workflow in OnlineCA.jl.

Example

CA can be easily reproduced on any machine where Julia is pre-installed by using the following commands in the Julia REPL window:

Installation

First, install OnlineCA.jl directly from GitHub:

```
# Install OnlinePCA.jl from the General registry
# (used for binary I/O preprocessing).
julia> Pkg.add("OnlinePCA")

# Install OnlineCA.jl from GitHub.
julia> Pkg.add(url="https://github.com/chiba-ai-med/OnlineCA.jl.git")
```

Preprocess of CSV

Then, write a synthetic data set as a CSV file and convert it to a compressed binary format using Zstandard. Matrix Market (MM) format is also supported for sparse matrices.

```
using OnlinePCA
using OnlinePCA: write_csv
using OnlineCA
using Distributions
using DelimitedFiles
using SparseArrays
using MatrixMarket

# CSV (input is assumed to be a non-negative integer count matrix)
tmp = mktempdir()
```

```
data = Int64.(ceil.(rand(NegativeBinomial(1, 0.5), 300, 99)))
data[1:50, 1:33] .= 100*data[1:50, 1:33]
data[51:100, 34:66] .= 100*data[51:100, 34:66]
data[101:150, 67:99] .= 100*data[101:150, 67:99]
write_csv(joinpath(tmp, "Data.csv"), data)

# Matrix Market (MM)
mmwrite(joinpath(tmp, "Data.mtx"), sparse(data))

# Binarization (Zstandard)
csv2bin(csvfile=joinpath(tmp, "Data.csv"),
        binfile=joinpath(tmp, "Data.zst"))

# Sparsification (Zstandard + MM format)
mm2bin(mmfile=joinpath(tmp, "Data.mtx"),
        binfile=joinpath(tmp, "Data.mtx.zst"))
```

Plot settings

A subplots helper using PlotlyJS.jl produces side-by-side scatter plots of the first three principal components.

```
using DataFrames
using PlotlyJS

function subplots(out_ca, group)
    F = out_ca[1]
    data_left = DataFrame(ca1=F[:,1], ca2=F[:,2], group=group)
    data_right = DataFrame(ca2=F[:,2], ca3=F[:,3], group=group)
    p_left = Plot(data_left, x=:ca1, y=:ca2, mode="markers",
                  marker_size=10, group=:group)
    p_right = Plot(data_right, x=:ca2, y=:ca3, mode="markers",
                  marker_size=10, group=:group, showlegend=false)
    for p in (p_left, p_right)
        p.data[1]["marker_color"] = "red"
        p.data[2]["marker_color"] = "blue"
        p.data[3]["marker_color"] = "green"
    end
    p_left.data[1]["name"] = "group1"
    p_left.data[2]["name"] = "group2"
    p_left.data[3]["name"] = "group3"
    p_left.layout["title"] = "Component 1 vs Component 2"
    p_right.layout["title"] = "Component 2 vs Component 3"
    p_left.layout["xaxis_title"] = "ca-1"
    p_left.layout["yaxis_title"] = "ca-2"
    p_right.layout["xaxis_title"] = "ca-2"
    p_right.layout["yaxis_title"] = "ca-3"
    plot([p_left p_right])
end

group = vcat(repeat(["group1"], inner=100),
             repeat(["group2"], inner=100),
             repeat(["group3"], inner=100))
```

CA on dense binary input

This example performs CA on the dense Zstandard-compressed binary file (Figure 2). The standardized residual matrix is never formed explicitly: `ca` streams through the data on disk to compute the implicit operator products and extracts the top `dim` components with randomized SVD.

```
out_ca = ca(input=joinpath(tmp, "Data.zst"),
            dim=3, noversamples=5, niter=3, chunksize=100)

subplots(out_ca, group)
```

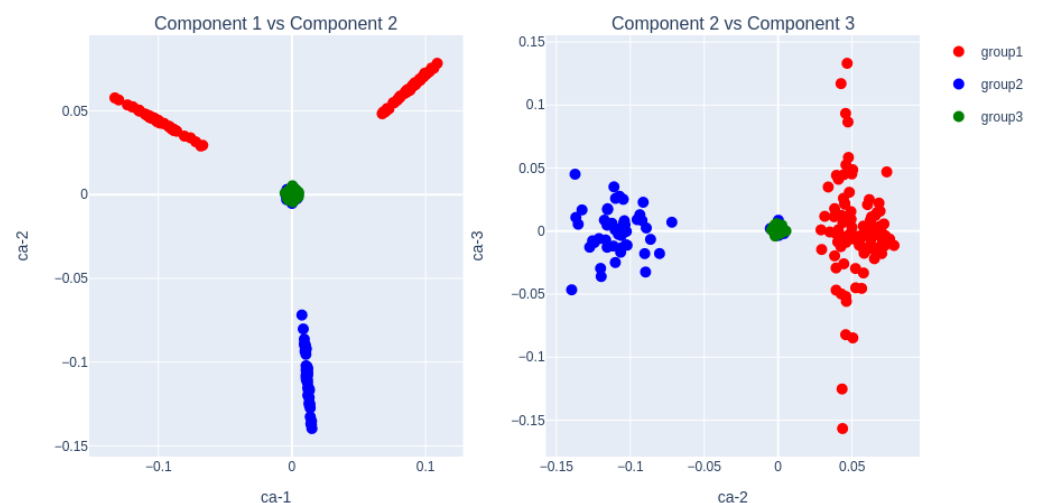


Figure 2: Output of `ca` against binarized CSV format.

Sparse CA on Matrix Market input

This example performs CA on the sparse MM input (Figure 3). `sparse_ca` reads (row, col, value) triplets in chunks and applies the same implicit operator without ever materializing dense intermediates of size $N \times M$.

```
out_sparse_ca = sparse_ca(input=joinpath(tmp, "Data.mtx.zst"),
                          dim=3, noversamples=5, niter=3, chunksize=100)

subplots(out_sparse_ca, group)
```

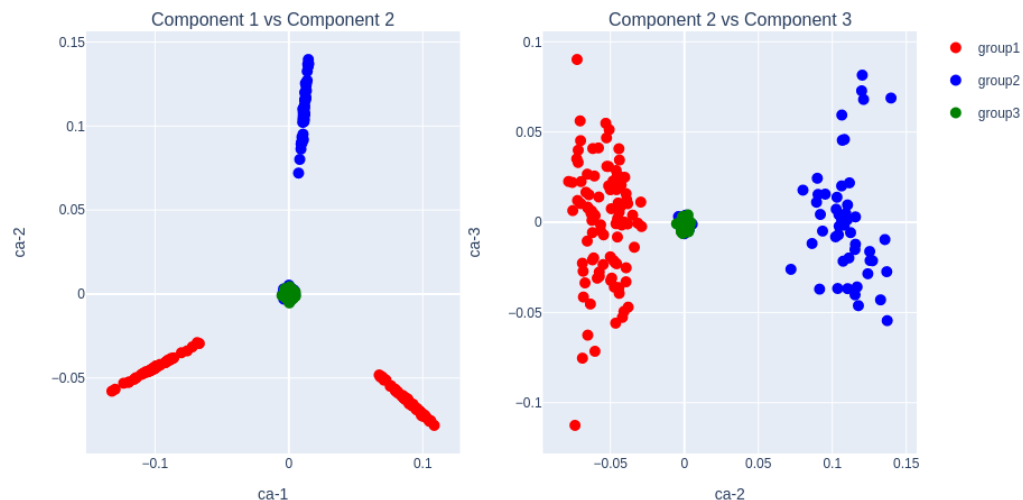


Figure 3: Output of `sparse_ca` against binarized MM format.

Multiple Correspondence Analysis (MCA)

MCA generalizes CA to a categorical table with q variables by applying CA to its $N \times M$ complete disjunctive (indicator) matrix, where each row has exactly q ones. `OnlineCA.jl` materializes the indicator on disk in BinCOO format and reuses `bincoo_ca`, so all out-of-core, sparse, and randomized-SVD machinery applies unchanged. The CA spectrum on an indicator matrix is artificially inflated; the standard per-axis correction

$$\lambda_k^* = \left(\frac{q}{q-1} \left(\lambda_k - \frac{1}{q} \right) \right)^2 \quad \text{if } \lambda_k > \frac{1}{q}, \text{ else } 0$$

is selectable through the correction keyword (Benzécri (Benzécri, 1979) vs. Greenacre (Greenacre, 2017)).

```
# 200 observations × 4 categorical variables, each with 3 levels
table = rand(1:3, 200, 4)
```

```
out_mca = mca(table; dim=3, correction=:benzecri,
               var_names=["age", "sex", "region", "occupation"])
# out_mca.inertia_adjusted, out_mca.total_inertia_adjusted, ...
```

A reference comparison against R's `ca` package on the classic Greenacre (Greenacre, 2017) smoke dataset is included in the test suite. See the README at <https://github.com/chiba-ai-med/OnlineCA.jl> for the BinCOO variant and supplementary projection examples.

Related work

There are various implementations of CA / MCA (Esteves, 2017; Halford, 2024; Lê et al., 2008; Nenadić & Greenacre, 2007; Oksanen & others, 2024) and some of them support sparse data formats (DeBruine, 2023; Hsu & Culhane, 2023), but `OnlineCA.jl` is the only tool that supports both OOC computation and language-agnostic sparse formats (MM and BinCOO), enabling seamless integration with external data pipelines.

Function Name	Language	OOB	Sparse Format	MCA
MASS::corresp	R	No	-	No
ca::ca	R	No	-	No
ca::mjca	R	No	-	Yes
FactoMineR::CA	R	No	-	No
FactoMineR::MCA	R	No	-	Yes
vegan::cca	R	No	-	No
prince.CA/MCA	Python	No	-	Yes
mca	Python	No	-	Yes
corral	R	No	dgCMatix	No
singlet	R	No	dgCMatix	No
OnlineCA.jl	Julia	Yes	MM/BinCOO	Yes

AI usage disclosure

Anthropic's Claude (Opus 4.6 and 4.7, accessed through the Claude Code CLI) assisted with drafting code, tests, docstrings, and prose across `src/`, `test/`, `paper/`, `README.md`, and `docs/`, including the phased refactoring to a `NamedTuple` result, `RNG` / seed threading, `Float32` support, MCA with the Benzécri / Greenacre correction, and supplementary projection. The author reviewed, edited, validated, and tested every AI-assisted output, and made all core design decisions — the implicit-operator decomposition, API contract, MCA's indicator-matrix route, and the output schema.

References

- Benzécri, J.-P. (1979). Sur le calcul des taux d'inertie dans l'analyse d'un questionnaire. *Les Cahiers de l'Analyse Des Données*, 4(3), 377–378.
- Braak, C. J. F. ter. (1986). Canonical correspondence analysis: A new eigenvector technique for multivariate direct gradient analysis. *Ecology*, 67(5), 1167–1179. <https://doi.org/10.2307/1938672>
- DeBruine, Z. J. (2023). *singlet: Non-negative matrix factorization and correspondence analysis for single-cell genomics*. <https://github.com/zdebruine/singlet>.
- Esteves, E. (2017). *mca: Multiple correspondence analysis in Python*. <https://github.com/esafak/mca>.
- Greenacre, M. (2017). *Correspondence analysis in practice* (3rd ed.). Chapman; Hall/CRC. <https://doi.org/10.1201/9781315369983>
- Halford, M. (2024). *prince: Multivariate exploratory data analysis in Python*. <https://github.com/MaxHalford/prince>.
- Halko, N., Martinsson, P. G., Shkolnisky, Y., & Tygert, M. (2011). An algorithm for the principal component analysis of large data sets. *SIAM Journal on Scientific Computing*, 33(5), 2580–2594. <https://doi.org/10.1137/100804139>
- Halko, N., Martinsson, P. G., & Tropp, J. A. (2011). Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2), 217–288. <https://doi.org/10.1137/090771806>
- Hill, M. O. (1974). Correspondence analysis: A neglected multivariate method. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 23(3), 340–354. <https://doi.org/10.2307/2347127>

- Hsu, L. L., & Culhane, A. C. (2023). Correspondence analysis for dimension reduction, batch integration, and visualization of single-cell RNA-seq data. *Scientific Reports*, 13(1), 1197. <https://doi.org/10.1038/s41598-022-26434-1>
- Lê, S., Josse, J., & Husson, F. (2008). FactoMineR: An R package for multivariate analysis. *Journal of Statistical Software*, 25(1), 1–18. <https://doi.org/10.18637/jss.v025.i01>
- Nenadić, O., & Greenacre, M. (2007). Correspondence analysis in R, with two- and three-dimensional graphics: The ca package. *Journal of Statistical Software*, 20(3), 1–13. <https://doi.org/10.18637/jss.v020.i03>
- Oksanen, J., & others. (2024). *vegan: Community ecology package*. <https://CRAN.R-project.org/package=vegan>. <https://doi.org/10.32614/CRAN.package.vegan>
- Tsuyuzaki, K. (2025). *OnlineNMF.jl: A Julia package for out-of-core and sparse non-negative matrix factorization*. <https://github.com/rikenbit/OnlineNMF.jl>. <https://doi.org/10.5281/zenodo.8199413>
- Tsuyuzaki, K. (2026). OnlinePCA.jl: A Julia package for out-of-core and sparse principal component analysis. *Journal of Open Source Software*, 11(117), 9343. <https://doi.org/10.21105/joss.09343>
- Tsuyuzaki, K., Sato, H., Sato, K., & Nikaido, I. (2020). Benchmarking principal component analysis for large-scale single-cell RNA-sequencing. *Genome Biology*, 21(1). <https://doi.org/10.1186/s13059-019-1900-3>